```
Step 7/7 : CMD ["bin/rails", "s", "-b", "0.0.0.0"]
 ---> Using cache
 ---> 577a1a5a2d2c
Successfully built 577a1a5a2d2c
Successfully tagged railsapp:latest
```

The image should build very quickly. If you look at the output, you'll see that for each step (other than the FROM instruction), it explicitly says Using cache. This indicates that Docker didn't need to create a new image for that step; it simply reused an intermediate image cached from a previous build.

The cache for a given step is invalidated when you modify the Dockerfile instruction to something that hasn't been built before. Additionally, COPY instructions can have their cache invalidated if copied files have changed since the step was last built. The comparison is made by the Docker daemon based on files in the build context—so ignoring irrelevant files in your .dockerignore file can also prevent cache invalidations.

Since a Dockerfile is sequential, with each instruction building on the previous (intermediate) image, when the cache is invalidated for one step, Docker must rebuild *every subsequent step*. It therefore hurts more when an earlier step is invalidated, because Docker has more steps to rebuild.

Understanding this is useful for keeping our image builds fast as we develop, without taking unnecessary *hits* from busting the image build cache.

In fact, our Dockerfile already has a slight issue with how the caching works…

Caching Issue 1: Updating Packages

Currently, our Dockerfile has the following two lines:

```
RUN apt-get update -yqq
RUN apt-get install -yqq --no-install-recommends nodejs
```

Although this works, there's a hidden problem lurking. Let's say we come along at a later stage and realize we need to install an extra package—for example, the Vim editor. We add the vim package to the apt-get install RUN instruction, busting the cache and causing that instruction to be rerun:

```
RUN apt-get update -yqq
RUN apt-get install -yqq --no-install-recommends nodejs vim
```

However, the apt-get update RUN instruction remains unchanged, and the cached repository details will be used. Rather than getting the current, latest version of the new package we've added, we'll be getting whatever was the latest *at the time we last built our image*. That behavior is almost never what we want.